Click Cheat Sheet

Click 8.5+ · `pip install click` · `import click`

MINIMAL COMMAND

```
@click.command()
@click.argument("name")
@click.option("--count", "-c", default=1,
    show_default=True, help="Repeats.")
def hello(name, count):
    """Greet NAME."""   # = --help text
    for _ in range(count):
        click.echo(f"Hello {name}")

if __name__ == "__main__":
    hello()
```

GROUPS & SUBCOMMANDS

```
@click.group()
@click.option("-v", "--verbose", count=True)
@click.pass_context
def cli(ctx, verbose):
    ctx.ensure_object(dict)
    ctx.obj["verbose"] = verbose

@cli.command()       # name "sync"
@click.pass_obj      # gets ctx.obj
def sync(obj): ...

cli.add_command(other, name="other")
```

`invoke_without_command=True`	Group runs alone; check `ctx.invoked_subcommand`
`chain=True`	`cli a b c` runs several
`@cli.result_callback()`	Post-process return values
`@click.command("my-name")`	Override name

`_` becomes `-`: `def dry_run` → `dry-run`.

ARGUMENTS (POSITIONAL)

Required	`@click.argument("src")`
Optional	`argument("dst", required=False)`
Variadic	`argument("files", nargs=-1)` → tuple
Fixed count	`argument("pt", nargs=2, type=float)`
From env	`argument("src", envvar="SRC")`

Arguments take `help=` since 8.5; otherwise describe them in the docstring.

OPTIONS

Value + default	`option("--n", default=1)` (type inferred)
Required	`required=True`
Flag	`option("--force", is_flag=True)`
On/off pair	`option("--color/--no-color", default=True)`
Repeatable	`multiple=True` → tuple
Counter `-vvv`	`count=True`
Feature switch	2 options, same dest, `flag_value="upper"/"lower"`
Tuple value	`nargs=2` or `type=(str, int)`
Env var	`envvar="APP_TOKEN"`, or `cli(auto_envvar_prefix="APP")`
Ask if missing	`prompt=True` / `prompt="Your name"`
Password	`hide_input=True`, `confirmation_prompt=True` or `@click.password_option()`
Help display	`show_default=True`, `hidden=True`
Python name	`option("--from", "from_")`
Deprecate	`deprecated=True` or `="use --x"`

Built-ins: `@click.version_option()` (auto-detects package version), `@click.confirmation_option(prompt="Sure?")`, `@click.help_option("-h", "--help")`, or `context_settings={"help_option_names": ["-h", "--help"]}`.

TYPES

`str int float bool`	Plain Python types work
`Choice(["a","b"], case_sensitive=False)`	Also takes an `Enum`
`IntRange(0, 10, clamp=True)`	Bounds; also `FloatRange`
`Path(exists=True, dir_okay=False, path_type=pathlib.Path)`	Validates, doesn't open
`File("rb")`, `File("w", lazy=True)`	Opens; `-` = stdin/stdout
`DateTime(["%Y-%m-%d"])`, `UUID`	→ `datetime`, `uuid.UUID`

Custom: subclass `click.ParamType`, set `name`, implement `convert(value, param, ctx)`, call `self.fail(msg, param, ctx)` on bad input.

VALIDATION CALLBACK

```
def check(ctx, param, value):
    if value % 2:
        raise click.BadParameter("even only")
    return value

@click.option("--n", type=int,
              callback=check)
```

`is_eager=True` + `expose_value=False`: process first, keep out of function args (how `--version` works).

OUTPUT & INPUT

Print	`click.echo(msg)`; stderr: `err=True`
Color	`click.secho("ok", fg="green", bold=True)`, `click.style()`
Ask	`click.prompt("Port", type=int, default=8080)`
Yes/no	`click.confirm("Continue?", abort=True)`
Pager	`click.echo_via_pager(text)`
Editor	`click.edit("initial text")`
Progress	`with click.progressbar(items, label="Work") as bar:`
Open file/URL	`click.launch(path_or_url)`
Config dir	`click.get_app_dir("myapp")`

ERRORS & EXIT CODES

`ClickException(msg)`	`Error: msg`, exit 1
`UsageError(msg)`	usage + error, exit 2
`BadParameter(msg)`	names the bad param, exit 2
`Abort()`	`Aborted!`, exit 1
`ctx.exit(code)`	exit with `code`

TESTING

```
from click.testing import CliRunner

def test_sync(tmp_path):
    r = CliRunner().invoke(
        cli, ["sync", str(tmp_path)],
        env={"APP_TOKEN": "t"}, input="y\n")
    assert r.exit_code == 0, r.output
    assert "done" in r.stdout
    # also r.stderr, r.output
```

`r.exception` holds an uncaught exception. Use `tmp_path`; `isolated_filesystem()` is deprecated in 8.5.

PACKAGING & COMPLETION

```
# pyproject.toml
[project.scripts]
mytool = "mytool.cli:cli"
```

Completion (bash, zsh, fish, powershell): `_MYTOOL_COMPLETE=zsh_source mytool > ~/.mytool-complete.zsh`, then source it from your shell rc.

GOTCHAS

- Parameter names must match function args (`--dry-run` → `dry_run`).
- Only one `nargs=-1` argument per command; it yields `()` when nothing is passed.
- `default` on a `multiple=True` option must be a list/tuple.
- `ctx.obj` is `None` until set: call `ctx.ensure_object(dict)`.
- Use `click.echo`, not `print()`: it respects `CliRunner` capture and strips color when piped.